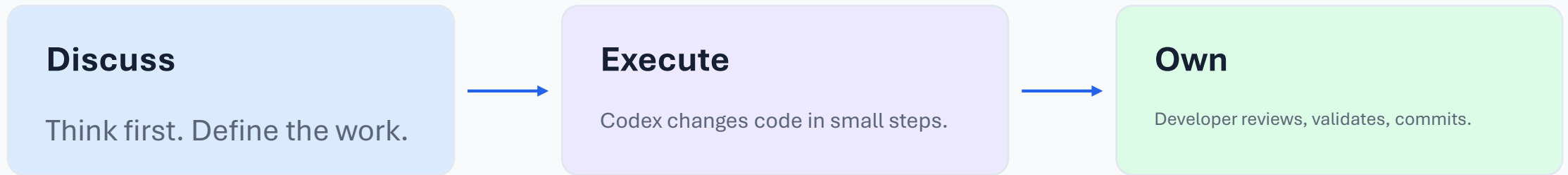


AI-Assisted Development Workflow

A simple process for thinking clearly, handing work to Codex, validating results, and repeating safely.



The AI accelerates the work. The developer controls the decisions.

The model: three roles, one loop

Do not ask Codex to implement until the work is understood.

Discussion Thread

Explore the problem, architecture, scope, risks, and acceptance criteria. This is where thinking happens.

Codex / Agent

Reads the handoff, confirms understanding, executes small changes, and runs validation commands.

Developer

Owens decisions, reviews the diff, validates behavior, commits locally, and captures follow-up work.

The simple flow

This is the process in one view.



The Markdown handoff is the bridge between discussion and execution.

Steps 1-2: Discuss and define the work

Start with a normal AI discussion thread. Do not touch code yet.

Start a discussion thread

Use the thread to explore the problem, options, risks, assumptions, and likely approach.

Track the purpose of the thread so tangents do not erase the main goal.

Define the work

Capture the goal, scope, out of scope, requirements, assumptions, acceptance criteria, risks, and testing expectations.

Step 3: Create the Codex handoff file

Turn the discussion into a Markdown artifact Codex can use.

The handoff should contain enough context for Codex to discuss and execute safely.

Include

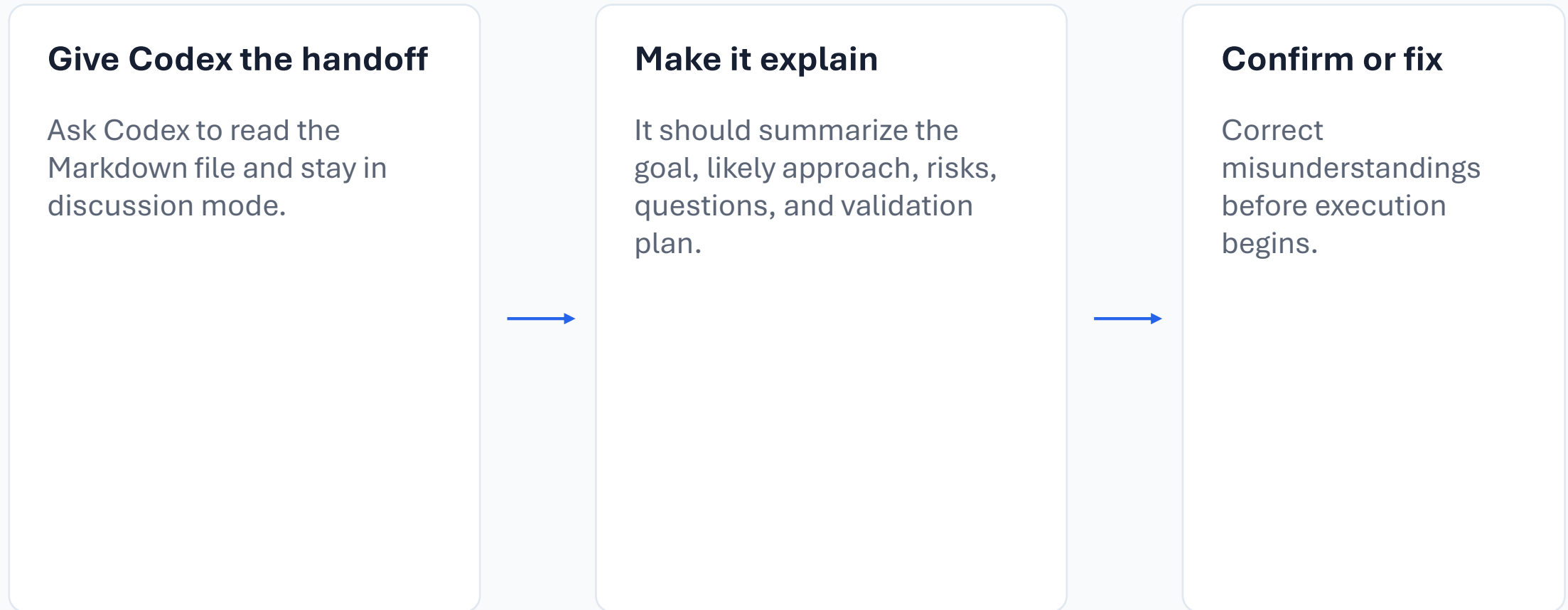
- Purpose
- Context
- Goal and scope
- Constraints
- Expected behavior

Also include

- Likely files or areas
- Implementation guidance
- Validation expectations
- Stop conditions
- Definition of done

Steps 4-6: Codex reads, explains, and gets corrected

The first Codex task is understanding, not implementation.



This checkpoint prevents plausible but wrong implementation.

Steps 7-8: Execute small and review hard

Keep every change small enough to understand.

Ask for one small step

Tell Codex to make the first meaningful implementation step only, then stop and summarize what changed.

Review the diff yourself

Check scope, architecture, patterns, complexity, behavior changes, and whether you understand the code.

Step 9: Validate at the right level

Passing commands are not enough. Know what each test proves.

Unit tests

Validate isolated logic.

Best for parsing, validation rules, helpers, and edge cases.

Smoke tests

Validate that the system or workflow still runs.

Best for pipelines and end-to-end execution.

Regression tests

Validate that expected behavior did not change accidentally.

Best for deterministic outputs.

Code Reviews based on the work completed.

Steps 10-12: Commit, summarize, repeat

Close each slice before starting the next one.

- ✓ Check in locally with a clear commit message.
- ✓ Summarize the thread if it contains reusable decisions or context.
- ✓ Capture files changed, tests run, validation limits, open questions, and follow-ups.
- ✓ Start the next task with the same rhythm.

Discuss → Define → Handoff → Confirm → Execute → Review → Validate → Commit → Summarize → Repeat

Simple explanation

Use this as the short version when explaining the process.

**The discussion thread is for thinking.
The Markdown handoff transfers clear intent.
Codex executes scoped work.
The developer reviews, validates, commits, and repeats.**

